

PredictiX — Code Review Report

Backend (FastAPI / Supabase) & Frontend (Next.js 16 / React 19)

Prepared: 3 July 2026 · Scope: full backend + frontend review · Report-only (no code changed)

What the project is

PredictiX is a **Smart Asset / Fleet Predictive-Maintenance system** for Sri Lankan warehouse vehicle fleets.

- **Backend** — FastAPI + SQLAlchemy on Supabase Postgres. ~40 routers, ML services (XGBoost failure classifier & regressor, Weibull survival models, HuggingFace seq2seq summarizers, a Groq/Llama agentic chatbot), APScheduler jobs (hourly batch predictions, daily service-reminder emails), and PDF report generation.
- **Frontend** — Next.js 16 + React 19 + Tailwind 4. Three roles (`user` , `admin` , `super_admin`) with localStorage-based JWT auth.

The functionality is broad and mostly works, but there are serious **security**, **correctness**, and **hygiene** issues detailed below, grouped by severity with file references.

Severity summary

Severity	Count	Theme
CRITICAL	6	Missing authentication, exposed debug data, LLM-SQL injection, committed secret, weak JWT fallback, hardcoded passwords
HIGH	8	super_admin logout, split DB config, None-session crashes, broken frontend build, dead middleware, hardcoded URLs, duplicate modules
MEDIUM	9	Silent excepts, print-logging, N+1 queries, open WebSocket, unpinned deps, CORS
LOW	6	Committed temp/log/model files, broken .gitignore, stray codemod scripts

1. Almost every backend endpoint is unauthenticated

Most routers depend only on `get_db`, never `get_current_user`. Anyone who can reach the API can read/modify all data with no token.

- `assets.py` — create/update/delete/list all assets (only `send-service-reminder` is gated).
`app/routers/assets.py:50`
- `users.py` — **create, update, delete users** and create Supabase auth accounts, no auth at all.
`app/routers/users.py:183, 273`
- `profile.py` — admin endpoints `GET /profiles`, `GET/PUT /profiles/{id}` are open → any anonymous caller can list every user and overwrite any profile. `app/routers/profile.py:37, 320`
- `user_profile.py` — `GET/POST /user-profile/users`, `PUT /user-profile/users/{id}` open.
`app/routers/user_profile.py:250`
- Also fully open: `warehouses`, `departments`, `maintenance`, `sensor_readings`, `reports`, `model_registry`, `asset_assignments`, `asset_documents`, `asset_status_history`, `ticket_comments`, `ticket_status_history`, `prediction_explanations`, `predictions`, `batch_predictions`, `survival_predictions`, `vehicle_predictions`, `admin_dashboard`, `warehouse_dashboard`, `asset_reports`, `asset_summaries`, `report_sources`, `notification_preferences`.

Fix: Add a shared auth dependency — e.g. `router.dependencies=[Depends(get_current_user)]` per router, plus explicit role checks on admin actions.

2. `db_debug.py` router mounted in production, dumps raw DB

`/debug/exact-db-values` and `/debug/asset-prediction-sample` have no auth and leak asset counts, health scores, IDs, ticket totals and cost sums. Registered in `main.py`.

`app/routers/db_debug.py:13` · `app/main.py:374`

Fix: Remove, or gate behind admin + an env flag.

3. LLM-generated SQL executed with prompt-only scoping

The chatbot `supabase_access` tool asks Groq/Llama to write raw SQL, then runs

`db.execute(text(sql_query))`. Row-level scoping for non-admins is enforced **only by appending an**

English instruction to the prompt ("you MUST scope..."). A user can get the model to ignore it and read other users' or all tickets/assets. The keyword blacklist (`INSERT|UPDATE|...`) is also bypassable.

`app/ai/agent/tools.py:64–105` · `scoping text at tools.py:263`

Fix: Enforce scoping in SQL/ORM (parameterised, server-side WHERE clauses), never in the prompt. Prefer a fixed set of parameterised queries over free-form LLM SQL.

4. Committed secrets

- `app/test_email.py` is tracked in git and contains a hardcoded Gmail app password (`zbvqirqvjzrwsuoc`) plus real addresses. `app/test_email.py:7`
- `.env` is not currently tracked (`.gitignore` has `.env*`) — good — but verify the backup `.bundle` copies in the parent directory don't contain it.

Fix: Rotate that Gmail credential now and delete the file from the repo.

5. Insecure JWT secret fallback

`get_current_user` and the auth router fall back to `os.getenv("JWT_SECRET", "supersecret")`. If `JWT_SECRET` is ever unset, tokens are signed/verified with a public constant → anyone can forge admin tokens. `core/config.py` requires the secret, but `deps.py` / `auth.py` bypass that Settings object and read env directly. [app/deps.py:48](#) · [app/routers/auth.py:44](#)

Fix: Read from the validated `settings` object and fail-fast if unset.

6. Hardcoded demo passwords + default-password login

`auth.py` hardcodes accounts like `super.admin1@lankalogix.lk` / `super`. Any profile without a stored hash logs in with `DEFAULT_PASSWORD` ("Predictix@123"). Fine for a demo, a liability in production. [app/routers/auth.py:86–117, 224](#)

HIGH Correctness

7. `super_admin` is locked out of all admin-only actions

Every role gate checks `role != "admin"` exactly. A logged-in `super_admin` (whom the frontend `isAdmin()` treats as admin) gets **403** on updating tickets, adding FAQs, sending reminders, etc. Role sets are inconsistent between frontend and backend.

[app/routers/assets.py:318](#) · [tickets.py:301](#) · [faqs.py:46](#)

Fix: Use a helper such as `role in {"admin", "super_admin"}`.

8. Two DB modules point at different databases

Module	Host	Port	Region
<code>app/db.py:21</code>	aws-0-ap-southeast-1	5432	Singapore
<code>app/db/session.py:23</code>	aws-1-ap-southeast-2	6543	Sydney

`app/db.py` is used by `Base`, models and most routers; `app/db/session.py`'s `SessionLocal` is used by the scheduler, dashboard-cache background workers, and `main.py`. So the request path and the background batch/cache path can talk to **different servers/regions/ports** — a latent split-brain bug.

Fix: Consolidate to a single engine / session factory.

9. `get_db()` can yield `None`, but most routers assume a `Session`

`deps.py` returns `None` when DB env vars are missing. Only a few routers handle it (dashboards check `if db is None`). The rest throw `AttributeError: 'NoneType' has no attribute 'query'` → 500s instead of a clean 503.

[app/deps.py:18](#)

Fix: Either always require the DB, or make every router defensive.

10. Frontend build is currently broken (5 TypeScript errors)

`next build` has no `ignoreBuildErrors`, so these fail the build:

- `AssetReportModal.tsx` (lines 321, 323, 325, 392) — `asset_name` / `asset_code` don't exist on the inferred type.
- `AssetFormDialog.tsx:183` — passing `duration` to a toast type that doesn't accept it.

The checked-in `tsc_errors.txt` lists an older, larger set (missing `AssetRecord`, `AssetHealthBand`, `AssetFilters`, a deleted page `@/components/Dashboard/WarehouseInsightsSection`) — partly fixed, now stale and misleading. Current real count is 5.

11. Dead Next.js middleware — client-only route protection

`src/proxy.ts` is a Next middleware but must be named `middleware.ts` to run. There is no `middleware.ts`, so it never executes. All route protection is client-side via `AuthGuard` / `RouteGuard` reading `localStorage`. Combined with finding #1 (open API), the app has effectively no server-side authorization anywhere.

`src/proxy.ts` · `src/components/auth/AuthGuard.tsx`

12. Hardcoded `http://127.0.0.1:8000` in production components

- `src/app/(admin)/admin/warehouse/report/page.tsx:38`
- `src/components/admin/warehouse/MaintenanceScheduleTable.tsx:23`

These bypass `NEXT_PUBLIC_API_URL` and will fail on Vercel. The `/api/proxy` route and `backendScanner.ts` port-scan `127.0.0.1:8000-8080` — a local-dev-only mechanism that cannot work in serverless deployment. The proxy layer appears half-abandoned (0 files import `/api/proxy`; 13 use `NEXT_PUBLIC_API_URL` directly).

13. Duplicate `ai_insight.py` / `ai_insights.py` (byte-identical)

The same 58-line file exists twice; only `ai_insights` is imported (`report_service.py:22`).

Fix: Delete `app/services/ai_insight.py`.

14. Two parallel profile/user router families that overlap and diverge

`profile.py` (`/profiles`) and `user_profile.py` (`/user-profile`) both implement "get me / update me / my assets / my stats / colleagues / list users / create user", with different code and different bugs.

`user_profile.py`'s `get_my_profile` swallows all errors and returns `{"error": ...}` with a 200 status; its `create_user` does **not** create a Supabase auth user (so the FK to `auth.users` fails or orphans), unlike `users.py` which does it correctly.

`app/routers/user_profile.py:91` · `users.py:194`

Fix: Consolidate; keep one implementation.

MEDIUM Reliability & design

- **Broad bare `except` / `except: pass`** hides failures across `user_profile.py`, `warehouse_dashboard.py`, `maintenance.py` — e.g. `user_profile.py:60` turns failures into silent zeros.
- **`print()` used for logging** throughout `user_profile.py`, `notifications.py:19` (a debug print on every call), `profile.py`. Use the existing `log`.
- **N+1 queries:** `profile.py` colleagues/assets loop a warehouse/departments lookup per row (`profile.py:290`); `user_profile.py` `get_all_users` runs per-user dept+warehouse+2 count queries. `users.py` `list_users` already does this correctly with pre-fetched maps — port that pattern.

- **Open WebSocket** — `/ws/notifications/{user_id}` trusts the path `user_id`, so anyone can subscribe to anyone's notification stream. `app/routers/websockets.py`
- **CORS** `allow_origins` + `allow_credentials=True` with a wide default list including localhost variants. Trim for production. `app/main.py:299`
- **Unpinned dependencies** — `requirements.txt` pins only `passlib/bcrypt`; `torch`, `transformers`, `fastapi`, `xgboost` unpinned → non-reproducible builds. Both `psycpg[binary]` and `psycpg2-binary` are listed.
- **Brittle severity inference** in `admin_dashboard.py:262` — recent-alert severity guessed by keyword-matching notification type strings.
- **JWT uses** `datetime.utcnow()` (deprecated in 3.12) and no `iat` / `nbf`. Minor.
- **Frontend smells** — 64 explicit `: any`, 10 stray `console.log`, JWT in `localStorage` (XSS-exposed; standard SPA tradeoff, but no CSP present).

LOW Repo hygiene / clutter

- **Tracked temp/log files:** `profile.out`, `uvicorn*.log` (4), `test_out.txt`, `admin_summary_tmp.json`, `temp_model.json` (9 MB!), `_check_maint_tmp.py`, `_temp_readme.md`.
- **.pk1 model weights committed** (classifier, regressor, 5 survival models, plus an `old/` folder of superseded ones). `.gitignore` only excludes `*.safetensors`.
- **Corrupted .gitignore**: last line is `s c r i p t s / s e e d _ f a q s . p y` (a space between every character — matches nothing).
- **src/script.py** — one-off Python codemod with a hardcoded `d:\Project\PredictiX-Frontend\...` path, sitting inside the Next.js `src`. Delete.
- **replace_urls.js** — one-off URL-rewrite codemod at frontend root. Delete or move to `scripts/`.
- Backend test scripts (`test_login_api.py`, `test_login_debug.py`, `test_models.py`, `test_hf.py`, `test_agent.py`) live in root rather than `tests/`. Duplicate project copies (`PredictiX_backend_clean`, `_BACKUP*.bundle`, `frontend backup`) in the parent dir — confirm no secrets inside those bundles.

Suggested priority order

1. **Add authentication to the backend** (#1, #2, #6) and **fix LLM-SQL scoping** (#3) — exploitable today.
2. **Rotate the leaked Gmail password and delete** `test_email.py` (#4).
3. **Fix the** `super_admin` **role gate** (#7) and **consolidate the two DB configs** (#8) — functional breakage.
4. **Fix the 5 TS errors / broken build** (#10) and decide on the proxy-vs-direct API strategy (#11, #12).
5. Clean up duplicates (#13, #14) and repo junk (Low section).